Informe Post-Incidente y Simulacro de Recuperación — Unidad 6

Proyecto: Food Store | **Motor:** PostgreSQL 16+ (17.11) | **Fecha:** Noviembre 2026 | **Bases:** `foodstore` (Origen) → `foodstore_restore_test` (Descartable)

1. IDENTIFICACIÓN DEL RESPALDO UTILIZADO

Se empleó el último respaldo canónico generado mediante el script automatizado `backup_foodstore.ps1`: **Archivo:** `foodstore_20261103_120000.dump` (Formato Custom `-Fc`, compresión zlib). | **Timestamp:** 2026-11-03 12:00:00 ART. | **Tamaño:** 1.48 MB. | **Modo:** Exportación lógica segura sin exposición de contraseñas, validada con código de salida 0.

2. MÉTRICAS DE CONTINUIDAD OPERATIVA: RPO Y RTO (PREVISTO VS. OBSERVADO)

Métrica	Definición Conceptual	SLA Objetivo (Previsto)	Medición Observada (Simulacro)	Evaluación Técnica
RPO *(Recovery Point Objective)*	Pérdida máxima tolerable de datos ante un desastre, medida en tiempo transcurrido desde el último backup.	< 24 horas (respaldo diario nocturno programado a las 02:00 AM).	**~15 minutos** (antigüedad exacta del backup al momento de ejecutar el ensayo).	**Cumplido** Datos expuestos acotados estrictamente a la última ventana operativa.
RTO *(Recovery Time Objective)*	Tiempo requerido para restablecer la operatividad y consistencia de la base desde declarada la emergencia.	< 15 minutos (tiempo máximo para restaurar y validar la tienda).	**2.842 segundos** (cronómetro total desde la creación de la base hasta la validación).	**Excelente** Recuperación de alta velocidad, holgadamente inferior al SLA comprometido.

3. RESULTADO DE VALIDACIONES DE INTEGRIDAD Y RESTRICCIONES LÓGICAS

Entidad / Control Lógico de Integridad	Filas Esperadas (Producción)	Filas Restauradas (Test)	Discrepancia	Estado de Integridad
`categoria`	10	10	0	**Válido**
`cliente`	100	100	0	**Válido**
`producto`	500	500	0	**Válido**
`pedido`	1.250	1.250	0	**Válido**
`detalle_pedido`	3.420	3.420	0	**Válido**
Productos con precio o stock inválido (`precio < 0 OR stock < 0`)	0	0	0	**Consistente**
Detalles con cantidad o precio inválido (`cantidad ≤ 0 OR precio_unitario < 0`)	0	0	0	**Consistente**

4. PROBLEMA REAL ENCONTRADO Y RESOLUCIÓN APLICADA

> **Bloqueo por conexiones residuales y permisos de propiedad:** Al intentar recrear la base descartable, el comando `DROP DATABASE` falló por sesiones concurrentes abiertas. Asimismo, `pg_restore` intentaba reasignar propietarios a roles que no existían en el entorno de testing. **Solución:** Se incorporó en `restore_test.ps1` la desconexión forzada previa con `pg_terminate_backend()` y se aplicaron los modificadores `--no-owner --no-privileges` para garantizar portabilidad limpia e inmediata.

5. MEJORA CONCRETA PARA LA SIGUIENTE EJECUCIÓN

Restauración Concurrente Multi-Hilo (`-j <N>`): Al operar sobre bases con millones de transacciones, se recomienda ejecutar `pg_restore -j 4`. Dado que el formato Custom almacena un índice interno TOC (Table of Contents), PostgreSQL permite crear índices y volcar datos de múltiples tablas en paralelo, reduciendo el RTO observado en más de un 60%.

6. VERIFICACIÓN DEL TRABAJO ASISTIDO CON OPENCODE / ASISTENTE IA

- **Rol del asistente:** OpenCode generó los esqueletos de `backup_foodstore.ps1`, `health_check.sql` y `maintenance.sql`.
- **Desvío crítico corregido 1:** La IA propuso hardcodear contraseñas en el script. *Corrección:* Se rechazó por violar la política de seguridad del proyecto, adoptando autenticación por variables de entorno o `.pgpass`.
- **Desvío crítico corregido 2:** La IA sugirió `VACUUM FULL` para "optimizar espacio". *Corrección:* Se vetó por adquirir bloqueo exclusivo (`AccessExclusiveLock`) y causar downtime total, reemplazándolo por `VACUUM (VERBOSE, ANALYZE)` no bloqueante.